```
obj = Object.new

class << obj
  include MyModule
end

obj.my_method            # => "hello"
obj.singleton_methods    # => [:my_method]
```

In case you think that opening the singleton class is a clumsy way to extend a class or an object, let's also look at an alternative technique.

Object#extend

Class Extensions (130) and *Object Extensions (130)* are common enough that Ruby provides a method just for them, named Object#extend:

class_definitions/module_trouble_extend.rb

```
module MyModule
  def my_method; 'hello'; end
end

obj = Object.new
obj.extend MyModule
obj.my_method       # => "hello"

class MyClass
  extend MyModule
end

MyClass.my_method   # => "hello"
```

Object#extend is simply a shortcut that includes a module in the receiver's singleton class. You can always do that yourself, if you so choose.

"Enough talking about singleton classes today," Bill announces. "I don't want to get a meta-headache. For now, let's go back to refactoring Bookworm."

Method Wrappers

Where you learn how to wrap a method inside another method—three different ways.

As the day draws to a close, you and Bill find yourselves stuck. Many methods in Bookworm rely on an open source library that retrieves a book's reviews from Amazon's website. The following code shows one example: